

ITA06 - Machine Learning

Exp :4 Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

Aim:

To build an Artificial Neural Network using the Backpropagation algorithm and test it using the XOR dataset.

Algorithm

1. Initialize weights and biases randomly.
2. Perform forward propagation.
3. Calculate the error.
4. Perform backpropagation to update weights.
5. Repeat until the error is minimized.
6. Test the trained network.

Program:

```
import numpy as np

# Input dataset (XOR)
X = np.array([[0,0],
               [0,1],
               [1,0],
               [1,1]])

# Expected output
y = np.array([[0],
               [1],
               [1],
               [0]])

# Sigmoid activation function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of sigmoid
```

```

def sigmoid_derivative(x):
    return x * (1 - x)

# Initialize weights and biases
np.random.seed(42)

input_neurons = 2
hidden_neurons = 2
output_neurons = 1

weights_input_hidden = np.random.uniform(size=(input_neurons, hidden_neurons))
weights_hidden_output = np.random.uniform(size=(hidden_neurons, output_neurons))
bias_hidden = np.random.uniform(size=(1, hidden_neurons))
bias_output = np.random.uniform(size=(1, output_neurons))

learning_rate = 0.5
epochs = 10000

# Training using Backpropagation
for epoch in range(epochs):
    # Forward Propagation
    hidden_input = np.dot(X, weights_input_hidden) + bias_hidden
    hidden_output = sigmoid(hidden_input)
    final_input = np.dot(hidden_output, weights_hidden_output) + bias_output
    predicted_output = sigmoid(final_input)

    # Error Calculation
    error = y - predicted_output

    # Backpropagation
    d_output = error * sigmoid_derivative(predicted_output)
    error_hidden = d_output.dot(weights_hidden_output.T)
    d_hidden = error_hidden * sigmoid_derivative(hidden_output)

    # Update weights and biases
    weights_hidden_output += hidden_output.T.dot(d_output) * learning_rate

```

```
bias_output += np.sum(d_output, axis=0, keepdims=True) * learning_rate
weights_input_hidden += X.T.dot(d_hidden) * learning_rate
bias_hidden += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate

# Output after training
print("Predicted Output:")
print(predicted_output.round(3))
```

```
print(predicted_output.round(3))
```

```
... Predicted Output:
[[0.019]
 [0.983]
 [0.983]
 [0.017]]
```

Result

The Artificial Neural Network was successfully implemented using the Backpropagation algorithm. The network was trained on the XOR dataset, and the predicted outputs closely matched the expected outputs, demonstrating successful learning.